

MANUTENÇÃO DE SOFTWARE

CONCEITOS INICIAIS E SISTEMAS LEGADOS

Débora Pelicano Diniz
deboradiniz@facef.br

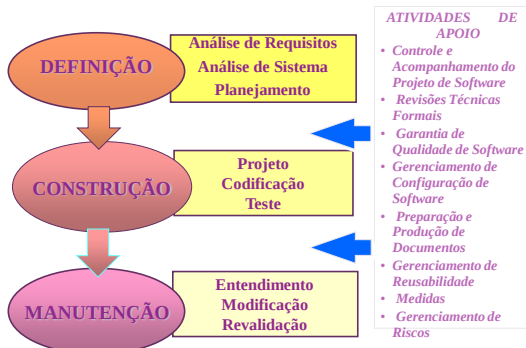
Contexto

- Um software nada mais é do que uma sequência lógica de comandos, então, se ele estiver correto, nunca mais precisará ser modificado.
- As empresas gastam muito dinheiro com sistemas, logo, eles devem durar muito.
- Sistemas são feitos para refletir comportamentos do mundo real, e o mundo real está em constante mudança.
- Se os sistemas não acompanharem essas mudanças, podem perder qualidade ou até deixarem de ser útil.
- Se o sistema tiver uma expectativa que não seja curta, deve ser construído visando facilitar a evolução.
- Para evoluir um software é necessário modificá-lo, ou seja, realizar algum tipo de manutenção seja em sua estrutura ou documentação.

Débora Pelicano Diniz

2

Fases genéricas do desenvolvimento de software



Débora Pelicano Diniz

3

Meir Lehman

- Nasceu em 1925 e morreu em 2010.
- Foi cientista e pesquisador na área da computação.
- Membro da Real Academia de Engenharia e professor na Escola de Ciências de Computação em Middlesex University.
- De 1972 à 2002 foi professor e Chefe do Departamento de Computação no Imperial College de Londres.
- Suas pesquisas contribuíram com o estudo do fenômeno da evolução do software.
- Propôs as Leis de Lehman da evolução do software.
- Trabalhou por anos na IBM.
- Sua frase: “no momento em que você instala um programa, o ambiente muda”.



Debora Pelicano Diniz

4

Meir Lehman

- Software que funciona no mundo real **nunca** será especificado de forma **definitiva**, pois
 - é impossível entender, ou prever, toda a complexidade do ambiente em que o software será usado e
 - porque, quando o software entrar no ambiente de uso, o ambiente será modificado pelo software.

Debora Pelicano Diniz

5

Os tipos de sistemas de Lehman

- Cada sistema possui uma forma diferente de evolução de acordo com sua natureza:
 - **S-Systems (specification)**: neste tipo de sistema o problema é bem definido e a solução é bem conhecida.
 - Está relacionado diretamente com o mundo real;
 - Se o mundo real muda, o resultado é um problema completamente novo que deve ser especificado, assim, são improváveis de mudar;
 - Um exemplo que se enquadra neste tipo de classificação são os sistemas para operações em matrizes.

Debora Pelicano Diniz

6

Os tipos de sistemas de Lehman

- Cada sistema possui uma forma diferente de evolução de acordo com sua natureza:
 - **P-Systems (problem)**: mais dinâmico que o S-Systems, baseia-se em uma **abstração** prática do problema, ao invés de uma especificação completamente definida.
 - Está sujeito a mudanças incrementais;
 - A solução produz informação que é comparada com o problema e depende, em partes, do analista que gerou os requisitos;
 - Os sistemas para jogo de xadrez são exemplos de sistemas desse tipo.

Debora Pelicano Diniz

7

Os tipos de sistemas de Lehman

- Cada sistema possui uma forma diferente de evolução de acordo com sua natureza:
 - **E-Systems (embedded)**: muda de acordo com as mudanças naturais do mundo real, já que está embutido no mundo real.
 - Como o sistema é uma parte do mundo modelado, está submetido às mudanças constantes;
 - Como exemplos, podem ser citados os sistemas para a área financeira.

Debora Pelicano Diniz

8

As Leis de Lehman

- O crescimento e a evolução de vários sistemas de grande porte foram examinados, de forma a entender como os softwares evoluem.
- Também foi analisado o processo de programação da IBM.
- Foram sendo definidas, desde o começo dos anos 70. Neste período foram formuladas as três primeiras leis.
- Na década seguinte foram apresentadas as duas seguintes e as três restantes vieram na década de 90.
- Estas leis se aplicam a qualquer software que resolva um problema ou implemente uma solução computacional no mundo real; estes sistemas são denominados sistemas do tipo E.

Debora Pelicano Diniz

9

As Leis de Lehman

- No século XX foram definidas as 8 leis da evolução de software para sistemas do tipo E (ou *E-Systems*), criadas por Meir Lehman, na década de 70.
- Sistemas do tipo E são os softwares desenvolvidos para resolver problemas do mundo real, operados por pessoas que se beneficiem deles em seu dia-a-dia.
- As Leis de Lehman eram generalizáveis e aplicáveis a diferentes contextos na tomada de decisão, planejamento, desenvolvimento e manutenção de software.
- As Leis de Lehman não falavam de pessoas, seguiam mais uma abordagem técnica.

Debora Pelicano Diniz

10

As Leis de Lehman

- Por 30 anos a Lei de Lehman esteve para o Software do século XX, assim como a Lei de Moore esteve para o hardware (duplicação da densidade de transistores a cada 2 anos).
- Mas o software deixou de ser algo restrito a grandes empresas e disseminou-se em todos os momentos da vida das pessoas.
- Até hoje as leis são válidas, elas não deixaram de ser provocativas, por gerarem reflexão em pontos relevantes no que diz respeito a pontos de atenção relacionados ao planejamento, construção e manutenção de sistemas.

Debora Pelicano Diniz

11

As Leis de Lehman

- **1. Mudança contínua** - Um software deve ser continuamente adaptado, senão, torna-se aos poucos cada vez menos satisfatório. A cada alteração no ambiente em que ele roda que exija nele melhorias, não fazê-las o tornarão progressivamente menos satisfatório naquilo para o que foi construído.
- **2. Complexidade crescente** - Se não forem tomadas medidas para reduzir ou manter a complexidade de um software, conforme ele é alterado sua complexidade irá aumentar progressivamente. Deve haver um esforço para reduzir a complexidade final de um sistema enquanto este recebe alterações.

Debora Pelicano Diniz

12

As Leis de Lehman

- **3. Auto regulação** - O processo de evolução de software é auto-regulado próximo à distribuição normal, com relação às medidas de produtos e atributos de processos. Depois de um tempo, o grupo que implementa a evolução do sistema estabelecerá uma dinâmica que fará com que o esforço incremental gasto em cada nova versão permaneça constante durante a vida do sistema.
- **4. Conservação da estabilidade organizacional** - A velocidade de atividade global efetiva de um software em evolução deverá se manter invariável durante todo o ciclo de vida deste produto. O mix que é levado em consideração para as tomadas de decisão que levam a evolução de um software tendem a ser constantes.

Debora Pelicano Diniz

13

As Leis de Lehman

- **5. Conservação de familiaridade** - Durante a vida útil de um software em evolução, a taxa de mudanças tende a ser proporcional ao domínio que a equipe detém. A taxa de evolução de um software está intimamente ligado ao grau de familiaridade dentre os profissionais que o mantém.
- **6. Crescimento Contínuo** - Todo software deve ter o conteúdo funcional continuamente ampliado durante seu ciclo de vida para manter a satisfação dos seus usuários. O projeto inicial não consegue incluir absolutamente tudo o que é necessário e, progressivamente, precisará ser aumentado.

Debora Pelicano Diniz

14

As Leis de Lehman

- **7. Qualidade diminuindo** - Os softwares desenvolvidos para resolver problemas do mundo real se depreciam progressivamente se eles não receberem as mudanças necessárias para adaptar-se ao que acontece em seu ambiente operacional, durante todo o tempo de seu ciclo de vida útil.
- **8. Sistema de feedback** - Os processos de manutenção e evolução de um software refletem sistemas de feedback em múltiplos níveis, *loops* e agentes e devem ser assim tratados para manter-se significativos.

Debora Pelicano Diniz

15

Respondam!!!

- Quais fatores que levam a modificação de um software existente?

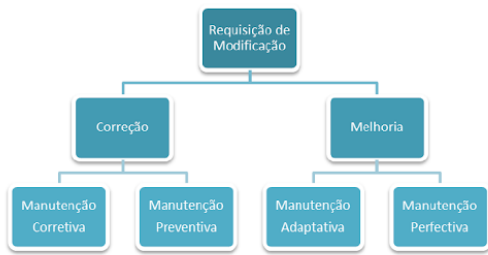
Razões para Modificações no Software

- Adicionar funcionalidades;
- Corrigir defeitos;
- Melhorar design;
- Comunicar com outros sistemas;
- Migrar de SO, BD, bibliotecas, entre outros;
- Adaptar a diferentes hardwares;
- Adaptar a leis, regras de negócio, entre outros;
- Refatorar código.

Manutenção de Software

- Alterações efetuadas no software **DEPOIS** de sua liberação, para que o software permaneça útil.
- Essas alterações ocorrem para:
 - Identificar e corrigir erros (**Manutenção Corretiva**);
 - Adaptar o software ao ambiente (**Manutenção Adaptativa**);
 - Atender pedidos do usuário para modificar funções existentes, incluir novas funções e efetuar melhoramentos gerais (**Manutenção Perfectiva**);
 - Melhorar a manutenibilidade ou confiabilidade futuras e fornecer uma base melhor para futuros melhoramentos (**Manutenção Preventiva**).
- Fase mais problemática do Ciclo de Vida de Software.
- Pode despendar mais de 70% de todo esforço de uma Organização.

Manutenção de Software

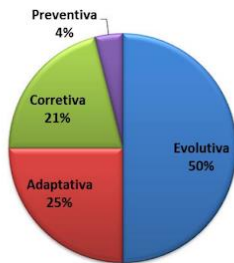


Debora Pelicano Diniz

19

Manutenção de Software

Proporção de ocorrências dos tipos de manutenção



Debora Pelicano Diniz

20

Manutenção de Software

- A princípio, as atividades de manutenção podem ser categorizadas individualmente, mas na prática, as atividades estão interligadas:
 - Ao modificar o código devido a uma nova biblioteca (adaptativa), defeitos podem ser introduzidos. Logo, esses defeitos devem ser corrigidos (corretiva);
 - A introdução de uma nova funcionalidade (evolutiva) pode requerer que código seja refatorado antes para facilitar sua implementação (preventiva).

Debora Pelicano Diniz

21

Manutenção de Software

- Por que é exigida tanta Manutenção? Por que é despendido tanto Esforço?
 - Existem softwares com mais de 10 anos no mercado:
 - Principal Interesse: Tamanho do Programa e Espaço de Armazenamento;
 - Nenhuma preocupação com a Arquitetura Global;
 - Sistemas mal estruturados;
 - Codificação, Lógica e Documentação ruins.
 - Esses sistemas devem continuar rodando e as alterações são inevitáveis:
 - Migração Para Novas Plataformas;
 - Melhoramentos Para Atender Novas Necessidades.

Debora Pelicano Diniz

22

Problemas com a Manutenção

- A maioria dos problemas é causada por deficiências na maneira como o software foi planejado e desenvolvido.
- Problemas clássicos:
 - É difícil ou impossível traçar a evolução do software por meio das várias versões. As alterações não são adequadamente documentadas;
 - É difícil ou impossível traçar o processo pelo qual o software foi criado;
 - É muito difícil entender programas **de outras pessoas**. A dificuldade aumenta conforme o número de elementos na configuração de software diminui;

Debora Pelicano Diniz

23

Problemas com a Manutenção

- Problemas clássicos:
 - Os "donos do software" frequentemente não estão presentes para explicar;
 - A documentação não existe, é incompreensível ou está desatualizada;
 - A maioria dos softwares não foram projetados para suportar alterações;
 - A manutenção não é vista como um trabalho glamoroso.

Debora Pelicano Diniz

24



Problemas com a Manutenção

- Problemas com o pessoal:
 - Entendimento limitado;
 - Prioridades de gerenciamento;
 - Ânimo.
- Problemas técnicos:
 - Recursos e paradigmas;
 - Dificuldade para a realização de testes.

Debora Pelicano Diniz

25



Responsabilidades da equipe de manutenção

- Entender o sistema;
- Localizar informações na documentação do sistema;
- Manter a documentação sistema atualizada;
- Ampliar as funções existentes, a fim de incluir novos requisitos ou requisitos modificados;
- Adicionar novas funções ao sistema;
- Descobrir a fonte das falhas ou problemas do sistema;
- Localizar e corrigir defeitos;
- Responder a questões sobre como o sistema funciona;
- Reestruturar o projeto e os componentes do código;
- Reescrever o projeto e os componentes do código;
- Excluir o projeto e os componentes do código que não são mais úteis;
- Gerenciar as mudanças que são feitas no sistema.

Debora Pelicano Diniz

26



Desenvolvimento x Manutenção

- Adicionar uma nova funcionalidade durante o desenvolvimento é mais “fácil” que durante a manutenção.
- Manutenção de software deve respeitar certos parâmetros e restrições existentes.
- Exemplo: ao projetar uma nova funcionalidade, o mantenedor deve investigar o sistema atual para abstrair sua arquitetura e detalhes de baixo-nível ;
 - Avaliar como a mudança acontecerá;
 - Prever o impacto da mudança (efeito cascata);
 - Determinar as características necessárias para o trabalho.

Debora Pelicano Diniz

27

Sistemas Legados

- Alguns sistemas que auxiliam na administração de uma empresa são usados há décadas.
- Eles foram incrementados com novas funcionalidades à medida que se tornam necessárias, e algumas delas não são mais relevantes e continuam fazendo parte do sistema por anos.
- Empresas que usam softwares que são responsáveis pela base da sua atividade são avessas a mudança, pois isso poderia afetar diretamente seu funcionamento, e acham que a segurança promovida por novos serviços não consegue chegar ao mesmo nível do que usam.
- Esses sistemas antigos que são fundamentais para a empresa são chamados de Sistemas Legados.

Debora Pelicano Diniz

28

Sistemas Legados

- Os sistemas que estão em uso hoje não são mais os mesmos que foram construídos há 10 ou 20 anos.
- Muitas pessoas se envolveram no processo de mudanças durante todos esses anos.
- Raramente existe uma especificação completa do sistema legado.
- Os processos corporativos e o modo como os sistemas legados operam estão intrinsicamente conectados.
- Importante regras corporativas podem estar presentes no sistema e não ser documentada em nenhum outro lugar.
- Realizar mudanças em sistemas legados é muito caro.

Debora Pelicano Diniz

29

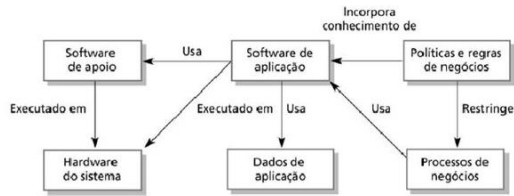
Estrutura dos Sistemas Legados

- Os sistemas legados não são apenas sistemas antigos. Eles incluem:
- **Hardware de sistema** – podem ser máquinas que nem estão mais disponíveis, com manutenção cara (Ex. Mainframe);
- **Software de apoio** – deste sistema operacional e utilitários até compiladores, que podem estar obsoletos e nem terem mais assistência técnica;
- **Software de aplicação** – fornece os serviços de negócios é composto de vários programas separados, desenvolvidos em diferentes épocas;
- **Dados de aplicação** – dados processados pelo sistema de aplicação, esses dados podem ser inconsistentes e podem estar duplicados em diferentes arquivos;
- **Processos de negócios** – os processos utilizados nas empresas a fim de atingir algum objetivo de negócios;
- **Políticas e regras de negócios** – definições de como a empresa deve ser conduzida e as restrições às quais ela deve se submeter.

Debora Pelicano Diniz

30

Componentes dos Sistemas Legados



(SOMMERVILLE, 2019)

Debora Pelicano Diniz

31

Modificações em Sistemas Legados

- Os Sistemas Legados podem ser vistos como sendo composto por camadas, em que cada camada depende e tem interface com a camada imediatamente inferior.



(SOMMERVILLE, 2019)

Debora Pelicano Diniz

32

Modificações em Sistemas Legados

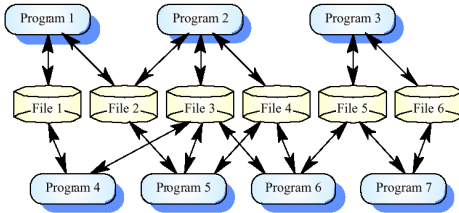
- Em tese, é possível fazer alterações em uma camada sem afetar qualquer uma das camadas adjacentes.
- Na prática, isso raramente funciona, as razões são:
 - A modificação de uma camada pode introduzir novos recursos e as camadas superiores podem ser modificadas para se beneficiarem desses recursos;
 - A modificação do software pode torná-lo mais lento, de modo que um novo hardware é necessário, a fim de melhorar o desempenho;
 - Muitas vezes, é impossível manter interfaces de hardware, especialmente se for proposta uma mudança radical para um novo tipo de hardware, por exemplo, mudar o hardware de mainframe para sistemas cliente-servidor.

Debora Pelicano Diniz

33

Modificações em Sistemas Legados

- O software de aplicação de um sistema legado não é um único programa e muito menos um único arquivo.



(SOMMERVILLE, 2019)

Debora Pelicano Diniz

34

Modificações em Sistemas Legados

- Mas muitos sistemas já sofreram a centralização do gerenciamento de dados em um sistema de Banco de Dados.
- Nesse caso, ficam duas questões importantes:
 - O sistema de gerenciamento de BD utilizado pode ser obsoleto e incompatível com os outros sistemas de gerenciamento de BD utilizados pela empresa;
 - O monitor de teleprocessamento utilizado pode ter sido projetado para uso com um determinado BD e mainframe. Pode não ser possível utilizar o mesmo monitor de teleprocessamento com um novo BD.

Debora Pelicano Diniz

35

Projeto de Sistemas Legados

- Muitos sistemas legados utilizados foram projetados antes do desenvolvimento orientado a objetos ser utilizado.
- Em vez de serem organizados com um conjunto de objetos interativos, esses sistemas são projetados utilizando uma estratégia de projeto orientado a funções.
- Uma modificação em uma função pode acarretar mudanças de comportamento em outras funções.
- Geralmente são modelados utilizando DFDs (Diagrama de Fluxo de Dados).

Debora Pelicano Diniz

36

Avaliação dos Sistemas Legados

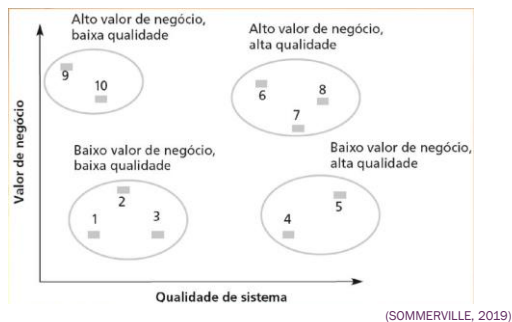
- As organizações que dependem de muitos sistemas legados precisam decidir como obter o melhor retorno de seu investimento:
 - Descartar completamente o sistema;
 - Continuar a manter o sistema;
 - Transformar o sistema de alguma maneira para melhorar sua facilidade de manutenção; ou
 - Substituir o sistema por um novo.
- A decisão deve considerar uma perspectiva de negócios (valor de negócios) e uma perspectiva de sistema (qualidade do sistema).

Debora Pelicano Diniz

37

Avaliação dos Sistemas Legados

- Existem quatro grupos de sistemas:



Debora Pelicano Diniz

38

Avaliação dos Sistemas Legados

1. **Baixa qualidade, baixo valor de negócios** - candidatos a serem descartados.
2. **Baixa qualidade, alto valor de negócios** - esses sistemas estão prestando uma importante contribuição à empresa, mas são dispendiosos para manter. São candidatos à transformação ou à substituição, se um sistema adequado estiver disponível.
3. **Alta qualidade, baixo valor de negócios** - não contribuem muito para o negócio, mas não são caros de serem mantidos. Podem ser descartados ou mantidos.
4. **Alta qualidade, alto valor de negócios** - Devem ser mantidos em operação, deve continuar a manutenção normal do sistema.

Debora Pelicano Diniz

39

Avaliação dos Sistemas Legados

- A avaliação pode ter diferentes pontos de vista que devem ser considerados. As pessoas relacionadas (*stakeholders*) devem ser entrevistadas e suas respostas devem ser registradas.
 - Tanto para a avaliação do valor do negócio quanto para a avaliação do processo de negócio.
 - Exemplos de perguntas:
 - Existe um modelo definido do processo e existem procedimentos implantados para verificar se esse modelo é seguido?
 - As diferentes partes da organização têm os mesmos processos para as mesmas funções?
 - Como o pessoal envolvido no processo adaptou o modelo a fim de torná-lo mais adequado a seu trabalho?
 - As relações com outros processos de negócios são necessárias? Se positivo, elas são claras para o pessoal envolvido no processo?
 - O processo é compatível com o software legado de aplicação? Ele fornece as informações requeridas? O processo exige que os mesmos dados sejam fornecidos diversas vezes, em diferentes lugares?

Debora Pelicano Diniz

40

Avaliação dos Sistemas Legados

- Avaliação da Qualidade do sistema:
 - Avaliação do processo de negócios - até que ponto os processos de negócios apoiam as atuais metas da empresa?
 - Avaliação do ambiente - como é a eficácia do ambiente de sistema e como é dispendioso para mantê-lo?
 - Avaliação do software de aplicação - qual é a qualidade do software de aplicação?

Debora Pelicano Diniz

41

Avaliação dos Sistemas Legados

- Avaliação do Ambiente:

Fator	Questões
Estabilidade do fornecedor	O fornecedor ainda existe? O fornecedor é financeiramente estável e deve continuar existindo? Se o fornecedor não está mais no negócio, existe alguém que mantém os sistemas?
Taxa de falhas	O hardware tem uma grande taxa de falhas reportadas? O software de apoio trava e força o reinício do sistema?
Idade	Quanto anos têm o hardware e o software? Quanto mais velho o hardware e o software de apoio, mais obsoletos serão. Ainda podem funcionar constantemente, mas poderão haver significativos benefícios econômicos e empresariais se migrassem para um sistema mais moderno.
Desempenho	O desempenho do sistema é adequado? Os problemas de desempenho têm um efeito significativo sobre os usuários do sistema?
Requisitos de apoio	Qual apoio local é requisitado pelo hardware e pelo software? Se houver altos custos associados a esse apoio, pode valer a pena considerar a substituição do sistema.
Custos de manutenção	Quais são os custos de manutenção de hardware e de licenças de software de apoio? Os hardwares mais antigos podem ter custos de manutenção mais elevados do que os sistemas modernos. Os softwares de apoio podem ter altos custos de licenciamento anual.
Interoperabilidade	Existem problemas de interface do sistema com outros sistemas? Compiladores podem, por exemplo, ser usados com as versões atuais do sistema operacional? É necessária a emulação do hardware?

(SOMMERVILLE, 2019)

Debora Pelicano Diniz

42

■ Avaliação do Software de Aplicação:

Fatores	Questões
Inteligibilidade	Quão difícil é compreender o código-fonte do sistema atual? Quão complexas são as estruturas de controle usadas? As variáveis têm nomes significativos que refletem sua função?
Documentação	Qual documentação do sistema está disponível? A documentação é completa, consistente e atual?
Dados	Existe um modelo de dados explícito para o sistema? Até que ponto os dados nos arquivos estão duplicados? Os dados usados pelo sistema são atuais e consistentes?
Desempenho	O desempenho da aplicação é adequado? Os problemas de desempenho têm um efeito significativo sobre os usuários do sistema?
Linguagem de programação	Compiladores modernos estão disponíveis para a linguagem de programação usada para desenvolver o sistema? A linguagem de programação ainda é usada para o desenvolvimento do novo sistema?
Gerenciamento de configuração	Todas as versões de todas as partes do sistema são gerenciadas por um sistema de gerenciamento de configuração? Existe uma descrição explícita das versões de componentes usadas no sistema atual?
Dados de teste	Existem dados de teste para o sistema? Existem registros dos testes de regressão feitos quando novos recursos forem adicionados ao sistema?
Habilidades de pessoal	Existem pessoas disponíveis com as habilidades necessárias para manter a aplicação? Existem pessoas disponíveis que tenham experiência no sistema?

(SOMMERVILLE, 2019)

- **Sistema de Medição** - é possível coletar dados quantitativos para ajudar a avaliar a qualidade do sistema de aplicação.
- Exemplos de dados quantitativos que podem ser coletados:
 - O número de pedidos de modificações no sistema;
 - O número de diferentes interfaces com o usuário utilizadas pelo sistema;
 - O volume de dados utilizados pelo sistema.

1. Explique por que os sistemas legados podem ser fundamentais para a operação de uma empresa.
2. Sob quais circunstâncias uma organização pode decidir descartar um sistema, quando a avaliação dele sugere que ele é de alta qualidade e alto valor de negócios?
3. Explique por que problemas com softwares de apoio podem significar que uma organização precisa substituir seus sistemas legados.
4. Que dificuldades podem surgir ao se ter diferentes componentes de um sistema legado implementados em diferentes linguagens de programação?
5. A gerência de uma organização pediu para você realizar a avaliação de um sistema e sugeriu que os resultados dessa avaliação mostrassem que o sistema está obsoleto e que deve ser substituído. Isso significará que uma série de mantenedores do sistema será redundante. Sua avaliação realmente mostra que o sistema tem uma boa manutenção e é de alta qualidade e alto valor de negócios. Como você relataria esses resultados à gerência?

■ BIBLIOGRAFIA BÁSICA:

- PFLEEGER, S. L. **Engenharia de software: Teoria e prática**. 2. ed. São Paulo: Prentice Hall, 2013.
- PRESSMAN, R. S. **Engenharia de software: Uma abordagem profissional**. 8. ed. Porto Alegre: AMGH editora, 2016.
- SOMMERVILLE, I. **Engenharia de software**.10. ed. São Paulo: Pearson Education do Brasil, 2019.
- WAZLAWICK, R. S. **Engenharia de software: Conceitos e práticas**. 2. ed. Rio de Janeiro: Elsevier, 2019.

■ BIBLIOGRAFIA COMPLEMENTAR:

- BOURQUE, P.; FAIRLEY, R. E. **Guide to the Software Engineering-Body of Knowledge**. 3. ed. New Jersey: IEEE Computer Society, 2014.
- HIRAMA, K. **Engenharia de software: qualidade e produtividade com tecnologia**. 1. ed. Rio de Janeiro: Elsevier, 2012.
- PAULA FILHO, W. P. **Engenharia de software: Fundamentos, métodos e padrões**. 3. ed. Rio de Janeiro: LTC, 2011.